ICE CREAM APP / PROGRAMACIÓN MÓVIL

Entiende tu proyecto. Defiéndelo con código.

Guía práctica de estudio y cambios en vivo

Preparada para tu proyecto Flutter + Node.js + PostgreSQL. Incluye respuestas para explicar el funcionamiento, rutas concretas para modificar el código y ejercicios con resultados esperados.

Lo más importante: antes de cambiar algo, identifica si estás ejecutando el modo demo o el conectado. El origen de los productos y el lugar donde se guardan los pedidos son diferentes.

Ruta de lectura

Páginas	Qué vas a practicar
2-4	Guion de presentación, recorrido de una compra y mapa de archivos.
5-7	Dart, Flutter, estado y cálculo de precios.
8-10	Cambiar precios, nombres, estilos, tamaños y toppings.
11-13	Agregar variables y un nuevo atributo persistente al producto.
14-17	Registro, base de datos, ejecución, pruebas y errores frecuentes.
18-19	Preguntas del profesor y simulacro final.

Cómo usarla

Primero lee la explicación; luego localiza el símbolo con **Ctrl + Shift + F** en VS Code. Practica un cambio a la vez, comprueba el resultado y explícalo en voz alta sin leer.

Base de revisión: código local consultado el 9 de noviembre de 2026. Los ejercicios son propuestas para practicar; este documento no modifica tu aplicación ni tus datos. Las rutas parten de la carpeta **ica.app**.

1. Tu presentación en 3 minutos

Un guion que puedes decir con tus palabras

Problema y objetivo. “Ice Cream App permite explorar helados, elegir tamaño y toppings, administrar un carrito y registrar pedidos. La interfaz está hecha con Flutter y Dart.”

Organización. “Separé la presentación, los modelos y el estado. El servidor está hecho con Node.js, Express y TypeScript; PostgreSQL guarda las cuentas y los pedidos del modo conectado.”

Una compra. “El usuario elige productos; la app calcula un total para mostrarlo. Al enviar el pedido, el servidor consulta los precios reales, vuelve a calcular el total y lo guarda asociado a la cuenta.”

Seguridad. “Las contraseñas se guardan como hash con sal. El servidor valida la sesión y los permisos. Un usuario normal solo consulta sus propios pedidos.”

Explica el estado real de la entrega

Parte	Lo que puedes afirmar
APK anterior	Es la entrega demo. Permite probar catálogo y pedidos simulados en el dispositivo.
Código actual	Contiene registro, login, pedidos por cuenta y administración. Incluye preparación para desplegar el servidor.
Pendiente de activar	Publicar la API HTTPS y compilar una APK conectada. El SDK configurado apunta a una unidad D: no disponible en esta revisión.
No implementado	Pagos, recuperación/verificación de correo, notificaciones y seguimiento de entrega. No se asigna tienda de recogida.

No digas “ya está en producción” si solo muestras la APK demo. Puedes decir: “La interfaz demo está instalada y el código conectado está implementado; voy a mostrar las partes que puedo ejecutar en este entorno”.

Cierra mostrando algo concreto

Abre **models.dart**, explica el cálculo de **CartLine.total** y realiza un cambio pequeño. Una explicación correcta acompañada de una prueba vale más que memorizar nombres de tecnologías.

2. Qué ocurre cuando hago un pedido

Paso	Quién lo hace y qué sucede
1. Interfaz	shop.dart muestra el catálogo. ProductPage y ToppingsPage recogen tamaño, extras y cantidad.
2. Estado	ShopStore.addCustomized agrega o agrupa una línea del carrito. notifyListeners avisa a la interfaz.
3. Cálculo	CartLine.unitPrice suma base + tamaño + toppings; total multiplica por cantidad. ShopStore.total suma las líneas.
4. Confirmación	CheckoutPage valida el formulario y llama a ShopStore.checkout.
5. Red	En modo conectado, POST /api/orders envía JSON, el token Bearer y una Idempotency-Key.
6. Servidor	requireUser obtiene el usuario de la sesión. validateOrder revisa los datos y priceOrder usa precios del servidor.
7. Persistencia	Una transacción guarda el pedido en ica_orders y el evento en ica_activity.
8. Respuesta	La API responde con el pedido. La app actualiza el historial y vacía el carrito tras confirmar el éxito.

Diagrama que puedes dibujar en el tablero

```
Pantallas Flutter
       | acciones / cambios visuales
       v
ShopStore + Product + CartLine
       | HTTP + JSON + token
       v
API Express -> validaciones -> PostgreSQL
       | respuesta del servidor
       v
Historial y confirmacion en Flutter
```

En demo: se omiten API y PostgreSQL para el pedido. Se construye un pedido con estado “Simulado” y se guarda localmente. No sirve para demostrar que un pedido llegó a una base remota.

Frase clave: “La app nunca se conecta directamente a PostgreSQL; se comunica con la API”.

3. Dónde buscar cada cambio

Archivo o carpeta	Responsabilidad / símbolo para buscar
frontend/lib/main.dart	Entrada main(), IceCreamApp, tema y CheckoutPage.
frontend/lib/models.dart	Product, demoProducts, sizes, toppings, money() y CartLine.
frontend/lib/store.dart	ShopStore: productos, carrito, favoritos, autenticación y llamadas HTTP.
frontend/lib/shop.dart	ShopHome, catálogo, productCard, carrito, historial, perfil y ProductPage.
frontend/lib/toppings_page.dart	Selección de toppings, filtros, resumen y ToppingPhoto.
frontend/lib/account.dart	AccountPage, formulario de registro/login y AdminPage.
frontend/lib/design.dart	Colores, ProductPhoto, productAsset y componentes visuales.
frontend/lib/sounds.dart	AppSounds y preferencia de audio.
frontend/pubspec.yaml	Dependencias, recursos, fuente y versión de la app.
backend/src/routes/	product.routes.ts, order.routes.ts, auth.routes.ts y admin.routes.ts.
backend/src/order.ts	validateOrder, priceOrder y precios de extras en el servidor.
backend/src/auth.ts	Hash de contraseñas y middleware de sesión/administración.
backend/schema.sql	Tablas, relaciones, índices y catálogo inicial.
backend/src/config/database.ts	Conexión a PostgreSQL mediante variables de entorno.
frontend/test / backend/test	Pruebas de comportamiento de la app y el servidor.

Cómo localizar: abre la carpeta ica.app en VS Code. Usa Ctrl + P para un archivo y Ctrl + Shift + F para un texto o símbolo. No dependas de números de línea: cambian al editar.

4. Dart y Flutter que debes explicar

Concepto	Ejemplo real y respuesta corta
Clase y objeto	Product es el molde; cada helado es un objeto con id, nombre y precio.
Constructor	Product(...) recibe los valores iniciales. this.id asigna el parámetro al atributo.
Tipos	int: cantidad o centavos; String: nombre; bool: bandera; List: colección ordenada; Map: pares clave-valor; Set: IDs únicos de favoritos.
final / const / var	final se asigna una vez; const exige un valor constante en compilación; var infiere el tipo y no significa “cualquier tipo”.
Null safety	String? puede ser null. ?? da un valor alternativo. ! afirma que no es null; si te equivocas, falla en ejecución.
Getter	int get total => unitPrice * quantity calcula un valor al consultarlo, sin mantener otra variable sincronizada.
factory / JSON	Product.fromJson transforma un Map de la API en un Product. CartLine.toJson crea un Map para enviar.
Widget	Es una descripción de una parte de la interfaz. build devuelve el árbol de widgets.
Stateless / Stateful	ProductPhoto recibe datos y no mantiene estado local mutable. ShopHome usa State para página, búsqueda y filtros.
Future / async / await	Una petición tarda. await espera su resultado sin bloquear la interfaz como una espera síncrona; no significa crear un hilo nuevo.
Listas y funciones	where filtra, map transforma, fold acumula. El carrito usa fold para sumar totales.

Explica esta línea sin memorizarla

```
int get total =>
    cart.fold(0, (sum, line) => sum + line.total);
```

“Empiezo en cero, recorro cada línea del carrito y acumulo su total. Si el carrito está vacío, el resultado es cero”.

final List no vuelve inmutable su contenido. En el proyecto, la referencia cart es final, pero puedes hacer cart.add(...) y cart.clear().

5. Estado y actualización de pantallas

Dos mecanismos distintos

setState: cambia información local de un State y pide reconstruir ese widget. Ejemplos: pestaña seleccionada, texto de búsqueda y mostrar u ocultar la contraseña.

ChangeNotifier: ShopStore comparte el estado de la tienda. Sus métodos modifican los datos y llaman a notifyListeners(). ListenableBuilder escucha esos cambios y reconstruye la parte correspondiente.

```
// Ejemplo conceptual del estado local:
setState(() {
  favoritesOnly = true;
});

// En ShopStore, despues de modificar el carrito:
notifyListeners();
```

Preguntas que suelen hacer

¿Por qué no guardar todo dentro de una pantalla? Porque el carrito y los pedidos se necesitan en varias pantallas. Centralizar esas operaciones reduce duplicación y permite probarlas sin tocar botones.

¿Puedo cambiar una variable sin setState? Sí, el valor cambia en memoria, pero la pantalla puede no reconstruirse. Usa setState para estado local o el método de ShopStore que notifica a sus oyentes.

¿Qué hace dispose? Libera recursos. Por ejemplo, se liberan los TextEditingController cuando se destruye la pantalla y se cierra el cliente HTTP del store.

¿Por qué revisar mounted después de await? Porque la pantalla puede haberse cerrado mientras esperabas la respuesta. Evita actualizar un State que ya no está montado.

Qué se conserva al cerrar

Dato	Dónde está en el modo conectado
Cuentas y pedidos	PostgreSQL. Se recuperan al iniciar sesión.
Sesión y carrito	Memoria. Un cierre completo requiere volver a iniciar sesión; el carrito es temporal.
Favoritos y sonido	Preferencias locales. Favoritos separados por cuenta; no se sincronizan entre celulares.

6. Precios: aprende las unidades

Regla: PostgreSQL products.base_price usa pesos COP. Flutter Product.price y los cálculos de pedido usan centavos enteros. $2.500 COP se representa como 2500 en la tabla y como 250000 en Dart.

Concepto	COP	Entero usado en cálculos
Base del ejemplo	2.500	250000
Extra dos bolas	1.000	100000
Oreo triturada	800	80000
Precio por helado	4.300	430000
Dos helados iguales	8.600	860000

Dónde ocurre la conversión

```
// Product.fromJson, en models.dart:
(num.parse("${j['price']}") * 100).round()

// priceOrder, en backend/src/order.ts:
const base = Math.round(Number(product.price) * 100);
```

El servidor convierte su precio base a centavos. Los extras ya están en centavos. money() convierte el entero en un texto legible con el símbolo y la etiqueta COP.

Fórmulas que debes poder explicar

```
unitPrice = base + extraTamano + sumaDeToppings
totalLinea = unitPrice * cantidad
totalCarrito = sumaDeTotalesDeLinea
```

¿Por qué enteros? Para que las sumas y multiplicaciones de los cálculos monetarios no dependan de aproximaciones decimales de punto flotante. La API convierte una vez y trabaja con enteros.

¿Por qué calcular dos veces? Flutter da una respuesta visual inmediata. El servidor decide el valor válido y rechaza un total manipulado o desactualizado.

Prueba mental: base $3.000 + dos bolas $1.000 + Oreo $800, cantidad 2 = **$9.600 COP**. En el pedido: 960000 centavos.

7. Reto: cambiar un precio

A. Si estás ejecutando demo

Archivo: frontend/lib/models.dart. Busca demoProducts y el producto por su id y nombre. Para subir Fresa de $2.500 a $3.000, cambia únicamente su precio final de 250000 a 300000.

```
Product(
  1,
  'Fresa de temporada',
  'Cremoso helado con fresas y un toque de vainilla.',
  'Clásicos',
  300000,
),
```

Comprueba: haz hot restart, abre Fresa y verifica base $3.000; con dos bolas y Oreo debe mostrar $4.800 por unidad. Usa un carrito nuevo para no confundir objetos cargados antes del cambio.

B. Si estás usando la API conectada

En pgAdmin, abre Query Tool de la base correcta. Primero identifica el producto; el id 1 es solo un ejemplo. Guarda el valor anterior para poder restaurarlo.

```
SELECT id, name, base_price FROM products ORDER BY id;

UPDATE products
SET base_price = 3000
WHERE id = 1
RETURNING id, name, base_price;
```

Comprueba: actualiza el catálogo deslizando hacia abajo. La consulta GET /api/products debe devolver price = 3000 y Flutter debe mostrar $3.000 COP. Envía un pedido nuevo y comprueba el total guardado.

> Cambiar demoProducts NO cambia PostgreSQL. Cambiar las semillas de schema.sql NO actualiza productos existentes: esas semillas solo se insertan si el catálogo está vacío.

Para revertir: restaura el entero anterior en Dart o usa UPDATE con el precio previo en pesos. Los pedidos históricos conservan el importe con el que fueron creados.

8. Reto: nombres, textos y colores

Cambiar el nombre de un helado

Demo: cambia el texto del Product en demoProducts. **Conectado:** actualiza products.name en la base. No cambies el id: es la identidad del producto.

```
UPDATE products
SET name = 'Fresa cremosa'
WHERE id = 1
RETURNING id, name;
```

Actualiza el catálogo y prueba la búsqueda con “cremosa”. Los pedidos antiguos guardan el nombre que tenían al crearse; renombrar el producto no reescribe su historial.

> Las imágenes actuales se eligen por palabras del nombre en design.dart → productAsset. “Fresa cremosa” conserva “fresa” y su imagen. “Frutilla” no coincide y puede mostrar la ilustración alternativa. Ajusta el mapeo si cambias esa palabra.

Cambiar un texto de la interfaz

Busca el texto exacto con Ctrl + Shift + F. Por ejemplo, “Tu momento más dulce” está en shop.dart. Cambia el literal, guarda y usa hot reload. Si el widget es const, el texto nuevo también debe ser constante para conservar const.

Cambiar el color principal

En frontend/lib/design.dart, modifica la constante violet. Ejemplo de cambio a un azul:

```
const violet = Color(0xff2563eb);
```

Color usa AARRGGBB: ff indica opacidad completa. Revisa botones, iconos y tema; algunos colores de decoración están escritos aparte y no dependen de violet.

Si te piden cambiar el nombre de la app

Distingue **texto visible dentro de Flutter**, **android:label** en android/app/src/main/AndroidManifest.xml y **applicationId** en android/app/build.gradle.kts. Cambiar el identificador no es solo renombrar: Android puede tratarla como otra app. Para cambiar la etiqueta instalada debes recompilar e instalar la APK.

9. Reto: tamaño, topping o producto

Subir “Dos bolas” de $1.000 a $1.200 extra

Cambia el valor a 120000 en **frontend/lib/models.dart → sizes** y en **backend/src/order.ts → sizes**. Los nombres deben coincidir exactamente en ambos lados.

```
// Dart: entrada dentro del mapa sizes
'Dos bolas': 120000,

// TypeScript: entrada dentro del objeto sizes
'Dos bolas': 120000,
```

Reinicia Flutter; recompila y reinicia el servidor. Prueba una base de $2.500, dos bolas y sin toppings: debe dar $3.700 COP. Si solo cambias el cliente, el servidor puede rechazar el total.

Agregar “Caramelo” por $600

```
// models.dart -> toppings
'Caramelo': 60000,

// models.dart -> toppingCategories
'Caramelo': 'Salsas',

// backend/src/order.ts -> toppings
'Caramelo': 60000,
```

El catálogo de extras usa toppingCategories: agregar solo el precio no basta para mostrar la opción. Revisa ToppingPhoto en toppings_page.dart si quieres una foto específica. No prometas una imagen nueva por escribir el nombre.

Comprueba: selección, resumen, precio por unidad, cantidad y pedido. El servidor permite como máximo nueve toppings seleccionados por línea; añadir una opción al catálogo no cambia ese límite.

Agregar un producto conectado

```
INSERT INTO products(name, description, category, base_price)
VALUES ('Fresa especial', 'Con trozos de fresa.',
        'Clásicos', 4000)
RETURNING id, name, base_price;
```

La base genera el id. Usa una categoría existente para esta práctica; una nueva puede necesitar un filtro adicional en shop.dart. En demo, añade otro Product con id único a demoProducts. Actualiza o reinicia según el modo.

10. Reto: agregar una variable local

Solicitud típica: “Agrega una variable que permita mostrar u ocultar un mensaje”. Este ejercicio cambia la interfaz, no necesita API ni base de datos.

Paso 1. Declara el estado

En frontend/lib/shop.dart, dentro de _ShopHomeState, junto a favoritesOnly, añade:

```
bool mostrarPromocion = true;
String mensajePromocional = 'Hoy es un buen día para un helado';
```

Paso 2. Úsalo en el catálogo

En el método catalog(), localiza la lista children del ListView dentro de RefreshIndicator. Inserta estos widgets antes de if (page == 0) hero():

```
SwitchListTile(
  title: const Text('Mostrar mensaje'),
  value: mostrarPromocion,
  onChanged: (value) {
    setState(() {
      mostrarPromocion = value;
    });
  },
),
if (mostrarPromocion)
  Padding(
    padding: const EdgeInsets.all(12),
    child: Text(mensajePromocional),
  ),
```

Paso 3. Demuestra y explica

Guarda, reinicia si acabas de añadir campos al State y mueve el interruptor. El mensaje debe aparecer y desaparecer. Explica que bool decide la visibilidad, String contiene el texto y setState pide redibujar.

> Al cerrar y abrir, el valor vuelve al inicial: es estado en memoria. Para conservarlo necesitarías persistencia, por ejemplo SharedPreferences. No uses const Text(mensajePromocional), porque el valor no es constante en compilación.

Reversión: quita los dos campos y los widgets añadidos. No cambia datos reales ni precios.

11. Reto: un atributo nuevo en Product

Solicitud: “Marca algunos productos como recomendados”. Un atributo del modelo describe a cada producto; no es lo mismo que una variable de la pantalla.

Paso 1. Amplía el modelo sin romper sus llamadas

En models.dart, añade el campo isFeatured y reemplaza el constructor por esta versión. El parámetro nombrado opcional permite que los Product(...) existentes sigan funcionando.

```
final bool isFeatured;

const Product(
  this.id,
  this.name,
  this.description,
  this.category,
  this.price, {
  this.isFeatured = false,
});
```

Paso 2. Lee el atributo desde JSON

En la llamada Product(...) de Product.fromJson, después del argumento que convierte el precio, agrega el argumento nombrado:

```
(num.parse('${j['price']}') * 100).round(),
isFeatured: j['is_featured'] as bool? ?? false,
```

false es el valor por defecto si una respuesta antigua no contiene la clave. El servidor debe enviar un booleano real, no el texto “true”.

Paso 3. Marca un producto demo

```
Product(1, 'Fresa de temporada', 'Helado de fresa.',
    'Clásicos', 250000, isFeatured: true),
```

Usa este formato para reemplazar un producto de prueba, no para crear accidentalmente otro con el mismo id. Haz hot restart y confirma que la app sigue compilando.

Hasta aquí el atributo existe en Dart. Para verlo en pantalla debes consumirlo; para guardarlo en el modo conectado debes ampliar también PostgreSQL y la respuesta de la API. Sigue en la página siguiente.

12. Conecta el atributo de extremo a extremo

Paso 4. Persiste la marca en PostgreSQL

```
ALTER TABLE products
ADD COLUMN IF NOT EXISTS is_featured BOOLEAN
NOT NULL DEFAULT FALSE;

UPDATE products SET is_featured = TRUE WHERE id = 1;
```

Ejecuta sobre la base de práctica. Añade también el ALTER TABLE a schema.sql antes de COMMIT, para que instalaciones futuras incorporen la columna. El UPDATE del ejemplo se aplica solo al producto que hayas elegido.

Paso 5. Incluye la columna en la API

En backend/src/routes/product.routes.ts, reemplaza el SELECT del catálogo por una cadena que incluya is_featured:

```
`SELECT id, name, description, category,
        base_price AS price, is_featured
 FROM products
 WHERE available = TRUE
 ORDER BY id`
```

Después ejecuta npm run build y reinicia el servidor. Si cambias solo TypeScript sin compilar, npm start sigue ejecutando dist, que puede contener la versión anterior.

Paso 6. Muéstrala en la tarjeta

En shop.dart → productCard(Product p), dentro de la Column del nombre y el precio, inserta después del widget Text(p.name, ...) completo:

```
if (p.isFeatured)
  const Text(
    'Recomendado',
    style: TextStyle(color: violet, fontSize: 11),
  ),
```

Revisa que la tarjeta no se desborde con nombres largos; si falta espacio, ajusta la altura de la tarjeta o usa una etiqueta sobre la imagen. Actualiza el catálogo para leer la marca nueva.

Prueba completa: columna true en la base → JSON con is_featured: true → Product.isFeatured true → etiqueta visible. Prueba también false y una respuesta sin la clave. No hace falta modificar el cálculo de pedidos: esta marca es informativa.

13. Registro, login y permisos

Explícalo siguiendo la petición

AccountPage valida nombre, correo y contraseña. ShopStore.authenticate envía POST /api/auth/register o /api/auth/login. El servidor normaliza el correo, verifica los datos y crea una sesión si la operación es válida.

Ruta	Para qué sirve
POST /api/auth/register	Crea una cuenta normal; no permite elegir rol administrador.
POST /api/auth/login	Comprueba credenciales y devuelve token + datos públicos del usuario.
GET /api/auth/me	Devuelve el usuario de una sesión válida.
POST /api/auth/logout	Elimina la sesión del servidor.
GET /api/orders	Consulta solo pedidos del usuario autenticado.
GET /api/admin/users	Consulta administrativa protegida; también existen orders y activity.

Respuestas para preguntas de seguridad

¿La contraseña está cifrada? “Más precisamente, está derivada con scrypt y una sal aleatoria. No guardo una contraseña recuperable. Comparo la derivación de lo que escribe el usuario con el hash almacenado”.

¿Dónde está el token? “En memoria de la app. El servidor guarda su hash y una fecha de expiración. La app lo envía en Authorization: Bearer. Al cerrar completamente la app se vuelve a pedir login”.

¿Basta con ocultar el botón Administración? “No. requireAdmin verifica el rol en el servidor. requireUser recupera la identidad de una sesión válida”.

¿Puedo enviar otro nombre o user_id? “El servidor toma la identidad de la sesión; el nombre del pedido viene de ese usuario, no de un campo libre del cliente”.

¿Está verificado el correo? “No. Actualmente identifica la cuenta, pero no hay verificación por correo ni recuperación de contraseña”.

14. Base de datos y consistencia

Tabla	Datos y relación
products	Catálogo: id, nombre, descripción, categoría, precio base y disponibilidad.
ica_users	Cuenta: UUID, nombre, correo único, hash de contraseña, rol y fechas.
ica_sessions	Hash de token, user_id que referencia a ica_users y vencimiento.
ica_orders	UUID, user_id, referencia de reintento, hash de solicitud, payload JSONB y fecha.
ica_activity	Usuario, tipo de evento, detalles JSONB y fecha.

Relaciones: un usuario puede tener muchos pedidos, sesiones y eventos. Las claves foráneas enlazan esos registros con ica_users.id. Los pedidos antiguos pueden no tener usuario; no pertenecen automáticamente a una cuenta nueva.

Consultas útiles para explicar, no para exponer claves

```
SELECT id, name, email, role FROM ica_users;

SELECT u.email, o.created_at,
       o.payload->>'status' AS estado,
       (o.payload->>'total')::numeric / 100 AS total_cop
FROM ica_orders o
LEFT JOIN ica_users u ON u.id = o.user_id
ORDER BY o.created_at DESC
LIMIT 10;
```

Tres conceptos que dan puntos

Transacción: BEGIN inicia; COMMIT confirma; ROLLBACK deshace si hay error. El pedido y su evento se guardan juntos para evitar una operación a medias.

Idempotencia: la misma referencia y el mismo contenido devuelven el pedido existente. Si cambia el contenido o el dueño, se rechaza. El bloqueo de transacción coordina reintentos simultáneos.

Consultas parametrizadas: WHERE user_id=$1 envía el dato separado del SQL. Es más seguro que concatenar entradas del usuario dentro de la consulta.

JSONB: el detalle conserva una fotografía de productos, nombres y precios del pedido. Una mejora futura sería separar cabecera y líneas en tablas si se requieren reportes relacionales más complejos.

15. Ejecutar y comprobar tus cambios

Antes de la clase

Confirma que la versión que muestras corresponde al código. Conserva una copia de práctica o usa control de versiones antes de editar. No subas .env, key.properties ni claves de firma al repositorio.

Modo demo: práctica de interfaz sin servidor

En una terminal situada en frontend, con Flutter y un dispositivo o emulador disponibles:

```
flutter devices
flutter run --dart-define=DEMO_MODE=true
```

Modo conectado local

Terminal 1, en backend, con PostgreSQL configurado en .env:

```
npm install
npm run db:setup
npm start
```

Terminal 2, en frontend, para un emulador Android:

```
flutter run --dart-define=API_URL=http://10.0.2.2:3000
```

10.0.2.2 es la dirección del computador desde el emulador Android. En un teléfono físico usa la IP local del computador y la misma red. localhost en el teléfono se refiere al teléfono. HTTP local se reserva al modo de depuración; distribución usa HTTPS.

Después de cada modificación

```
// Terminal en frontend (comandos, sin estas barras):
flutter analyze
flutter test

// Terminal en backend:
npm test
```

Los dos comentarios del bloque solo indican la carpeta. No necesitas escribirlos. Para incluir la prueba de integración PostgreSQL del backend, en PowerShell define **$env:TEST_DATABASE='1'** antes de npm test; utiliza un esquema temporal aislado.

Hot reload: útil para textos y diseño. **Hot restart:** vuelve a crear el estado y ayuda con datos iniciales o campos nuevos. Cambiar un dart-define requiere detener y volver a ejecutar. Una APK ya instalada no se actualiza solo por guardar un archivo.

16. Si algo falla durante la evaluación

Síntoma	Qué revisar primero
El precio no cambia	¿Demo o conectado? ¿Centavos o pesos? ¿Actualizaste catálogo o reiniciaste el estado inicial?
Cambié SQL y el historial no cambia	Es correcto: cada pedido conserva el detalle con el que fue creado. Comprueba con un pedido nuevo.
No aparece un topping	Debe existir en toppings y toppingCategories del cliente. El servidor debe conocer su nombre y precio.
El servidor rechaza el total	Compara base, tamaño, extras, cantidad y unidad monetaria en cliente y servidor. Actualiza el catálogo.
No se actualiza la pantalla	¿Se ejecutó setState o notifyListeners? ¿Modificaste la instancia que usa esa pantalla?
No se puede registrar	¿API_URL está definida? ¿Servidor y PostgreSQL funcionan? ¿Correo válido, clave de 10 a 128 caracteres y confirmación igual?
401 / 403	401: falta sesión o venció. 403: usuario autenticado sin permiso para esa operación.
400 / 409 / 429 / 503	400: entrada inválida. 409: conflicto. 429: demasiados intentos. 503: el servidor no pudo completar la operación.
La foto cambió al renombrar	productAsset usa palabras del nombre. Revisa el mapeo o la ilustración alternativa.
Compila TypeScript pero no veo cambios	npm start ejecuta dist. Recompila con npm run build y reinicia el proceso.
No puedo generar APK	Verifica Flutter, Java, SDK Android, local.properties y la firma. En esta revisión no está disponible la unidad D: configurada.

Método para responder bajo presión

1. Repite qué comportamiento debería verse. 2. Identifica la capa responsable. 3. Cambia una sola cosa. 4. Lee el mensaje de error. 5. Comprueba un caso normal y uno inválido. 6. Explica el resultado, incluso si descubres una limitación.

Si no sabes una respuesta, explica cómo la investigarías en el proyecto. No afirmes que una característica está implementada si no puedes señalar el código o demostrarla.

17. Preguntas rápidas del profesor

Arquitectura y programación

¿Por qué Flutter? Permite construir la interfaz con widgets y reutilizar buena parte del código entre plataformas. Este proyecto entrega Android; tener carpeta iOS no demuestra que se haya compilado en iPhone.

¿Usaste MVC? No lo presentaría como un MVC estricto. Hay separación entre modelos, pantallas, estado y servidor; el estado central está en ShopStore.

¿Qué es encapsulación? Agrupar datos y comportamiento y controlar su acceso. En Dart un nombre que comienza por _ es privado a su biblioteca; _token no es un campo público del store.

¿Qué hereda ShopStore? ChangeNotifier. Eso le permite mantener oyentes y avisarles con notifyListeners().

¿Por qué el id y no el nombre? El nombre puede cambiar o repetirse. El id identifica el producto para relaciones y pedidos.

Datos y operaciones

¿Esto es CRUD completo? No en todas las entidades. Hay creación y lectura de cuentas/pedidos, pero no una interfaz completa para editar y eliminar todo. El catálogo se consulta; los ejercicios de edición se hacen en código o SQL.

¿Qué pasa si se pierde internet? El modo conectado muestra el error y no se convierte en demo. Un fallo de envío sin confirmación conserva normalmente el carrito para reintentar; una sesión inválida limpia datos de la cuenta.

¿Puedo comprar sin cuenta? En conectado, el servidor exige sesión para crear pedidos. Demo permite practicar sin una cuenta real.

¿Cómo evitas duplicados? Con una referencia de idempotencia que se reutiliza al reintentar el mismo contenido dentro de la sesión. La base tiene una restricción única y se comprueba el propietario.

Calidad y mejoras

¿Cómo lo probaste? Hay pruebas Flutter de estado e interfaz y pruebas del backend. El informe del 31/10/2026 registra 14 de Flutter y 8 del servidor aprobadas. Ese resultado histórico no sustituye ejecutar las pruebas después de tus cambios.

¿Qué mejorarías? Publicar y probar el modo conectado en dispositivos reales, agregar recuperación y verificación de correo, mover los extras a un catálogo central y completar la administración de productos.

¿Usaste ayuda externa? Responde con honestidad según lo que hiciste y las reglas del curso. Demuestra comprensión localizando el código, explicando decisiones y realizando un cambio verificable.

18. Simulacro y hoja de repaso

Ensayo de 25 minutos

Tiempo	Reto	Resultado que debes demostrar
3 min	Presenta el proyecto	Objetivo, tecnologías y estado real de la APK.
4 min	Cambia Fresa a $3.000	Elige la fuente correcta y explica centavos/pesos.
3 min	Renómbrala	Búsqueda actualizada y foto revisada.
4 min	Crea una variable bool	Interruptor que modifica una parte visible.
5 min	Explica un pedido	Interfaz, store, HTTP, validación y transacción.
3 min	Justifica la seguridad	Hash, sesión y separación de cuentas.
3 min	Muestra una prueba	Caso válido, inválido y cómo restaurar el cambio.

Cinco frases para recordar

“Primero identifico de dónde vienen los datos”.
“Los precios internos se calculan en centavos”.
“La interfaz muestra; el servidor valida”.
“Cambiar el modelo no cambia automáticamente la base”.
“Guardar el archivo no actualiza una APK instalada”.

Lista antes de salir a la evaluación

Lleva el proyecto, esta guía, una APK que hayas probado y el cable de datos. Verifica Flutter, dispositivo y servidor antes de la clase. Ten una cuenta de prueba si vas a mostrar el modo conectado. Usa datos de práctica para cambios SQL y anota los valores originales.

Fuentes de esta guía

Lectura directa del proyecto local: frontend/lib/{main,models,store,shop,account,design,toppings_page,sounds}.dart; frontend/pubspec.yaml; backend/src/{auth,order,app}.ts; backend/src/routes/; backend/schema.sql; scripts/build-android.ps1 y los informes de entrega. Las rutas y los símbolos se verificaron el 09/11/2026.

Los fragmentos marcados como ejercicios son cambios propuestos para practicar, no funciones que ya estén agregadas. No se ejecutaron los UPDATE/ALTER del documento ni se modificó el código de la aplicación al preparar esta guía.